Phase 3: Build a Local Order Book and Core Trading Infrastructure

100 Points Possible

4/19/2026

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

Details

Phase 3: Build a Local Order Book and Core Trading Infrastructure

🎯 Purpose

You're building a **real-time trading system core** — not a market simulator, not an exchange.

This system needs to:

- Maintain a **real-time view of the market** via a local order book
- Track **your own submitted orders** and their state (working, filled, canceled)
- Provide a clean, memory-safe architecture using smart pointers and RAII
- Serve as the foundation for live trading with Alpaca in Phase 4

🚧 Why a Local Order Book?

A trading system must make decisions based on **what the market looks like** right now. You can't wait for confirmation from a broker or rely on a black-box price feed. Instead, your system needs to maintain:

✅ A **view of bid/ask prices and quantities**
✅ A structure that is **updated with each market event**
✅ Access to the **best available price levels** instantly

This is your **internal local order book** — it doesn't manage external orders; it reflects the public price levels you're trading against.

💡 Key Concepts You'll Apply

- Use of `std::unique_ptr`, `std::shared_ptr` for managing price level nodes
- Manual resource handling via RAII principles (constructors/destructors)
- Deallocation of orders after cancellation or fill
- Avoiding memory leaks and dangling pointers while dealing with live data
- Designing a system that runs safely and predictably under high-speed conditions

🔩 System Modules

In this phase, you're not just building isolated functions — you're designing a **cohesive, real-time trading engine**. Each module plays a critical role in allowing your system to perceive the market, make decisions, and act in a memory-safe and efficient way.

1️⃣ Market Data Structures – *Your Real-Time View of the Market*

This module is responsible for maintaining your system's **internal representation of market state** — typically just the best bid and ask (Level 1), but optionally deeper depth (Level 2).

🔧 Why this matters:

Your strategy needs **fast, local access** to:

- The best price to buy or sell
- How much liquidity is available at those prices
- How those values are changing over time

You'll use this snapshot to determine *if*, *when*, and *at what price* to place an order.

📁 `market_snapshot.h`

```
struct PriceLevel {
    double price;
    int quantity;

    PriceLevel(double p, int q) : price(p), quantity(q) {}
};
```

Here, each price level is dynamically managed using `std::unique_ptr`, so when a level is removed (e.g., quantity goes to zero), the memory is cleaned up automatically.

```
class MarketSnapshot {
    std::map<double, std::unique_ptr<PriceLevel>> bids; // sorted descending
    std::map<double, std::unique_ptr<PriceLevel>> asks; // sorted ascending
```

- `bids`: map of price → quantity, sorted **high to low**
- `asks`: map of price → quantity, sorted **low to high**

These maps give you **instant access to the best prices** — the top of book.

🔐 Smart pointer usage:

- Use `std::unique_ptr` for exclusive ownership of each price level
- No need to manually `delete` — RAII handles cleanup when a level is removed

📌 Interface examples:

```
void update_bid(double price, int qty);
void update_ask(double price, int qty);
const PriceLevel* get_best_bid() const;
const PriceLevel* get_best_ask() const;
```

These methods allow safe updates to your market state, and clean access for your strategy logic.

2️⃣ Order Manager – *Tracking Your Actions and Risk*

The order manager is responsible for **tracking all of the orders you've sent**, their status, and how much has been filled. It is your **memory-safe internal ledger**.

🔧 Why this matters:

- You need to know if you already sent an order at a certain price.
- You need to know if you're still waiting to be filled.
- You need to avoid placing conflicting or redundant orders.
- You must clean up resources properly when orders are no longer needed.

📁 `order_manager.h`

```
enum class OrderStatus { New, Filled, PartiallyFilled, Cancelled };

struct MyOrder {
    int id;
    Side side;
    double price;
    int quantity;
    int filled = 0;
    OrderStatus status = OrderStatus::New;
};
```

This structure tracks each of your live orders and is stored in a map:

```
std::map<int, std::unique_ptr<MyOrder>> orders;
```

🔐 Smart pointer usage:

- Orders are dynamically allocated and stored in `unique_ptr`
- When an order is filled or canceled, simply remove it from the map
- The memory is **automatically and safely reclaimed**

📌 Interface examples:

```
int place_order(Side side, double price, int qty);
void cancel(int id);
void handle_fill(int id, int filled_qty);
void print_active_orders() const;
```

Each order gets a unique ID, and you can fully track, modify, or remove any order cleanly.

3️⃣ Strategy Execution Loop – *Making Decisions and Sending Orders*

This is the glue that connects your **view of the market** and your **order actions**. It listens to market updates, makes decisions based on rules, and instructs the order manager to act.

🔧 Why this matters:

- This is where **your trading logic lives**
- It reacts to market changes and chooses when to buy, sell, or wait
- It directly calls the MarketSnapshot and OrderManager to carry out your decisions

📁 `main.cpp`

You'll typically run a loop like this:

```
for (auto& update : load_feed("sample_feed.txt")) {
    snapshot.update_bid(update.bid_price, update.bid_qty);
    snapshot.update_ask(update.ask_price, update.ask_qty);

    if (should_trade(snapshot)) {
        int id = om.place_order(Side::Buy, snapshot.get_best_bid()->price, 10);
        std::cout << "Placed BUY order at " << snapshot.get_best_bid()->price << "\n";
    }

    if (update.filled_order_id != -1) {
        om.handle_fill(update.filled_order_id, update.qty);
```

```
    }
}
```

This connects all parts together:

- The **MarketSnapshot** receives and stores market data
- The **OrderManager** places and updates orders
- The **Strategy Loop** coordinates decision-making

🔐 Memory safety:

- No `new` or `delete` required
- Everything is managed through `unique_ptr`, with clear ownership semantics
- You cannot accidentally delete an order that's still in use

✅ Required Behaviors

This phase introduces multiple interacting components that must behave predictably and reliably in real-time. The system should demonstrate the following:

---

1. **Market Snapshot Maintenance**

- Must continuously and correctly update the best bid and ask based on new incoming data.
- If quantity becomes zero, the price level should be **removed**.
- If a new price appears, it should be **added as a new price level**.

📌 *Example Behavior*:

```
Input:  BID 100.20 300
Output: Best bid is now 100.20 x 300

Input:  BID 100.20 0
Output: Best bid removed (next best price becomes top of book)
```

---

2. **Memory-Safe Price Level Tracking**

- Use `std::unique_ptr` to manage each `PriceLevel`.
- Ensure that each price level is deallocated when removed.

- Confirm that no raw pointers or manual delete operations are present in the codebase.

---

3. **Order Tracking & Lifecycle Management**

- Orders must be assigned unique IDs.
- Orders should be stored dynamically (e.g., `std::unique_ptr`) and removed when:
  - Completely filled
  - Canceled
- Must support:
  - Order placement
  - Manual cancellation
  - Handling execution updates

📌 *Example Behavior*:

```
Place: BUY 100.10 x 50 → ID = 1
Execution: Order 1 filled 30
Status: Partial fill
Execution: Order 1 filled 20
Status: Filled → Order removed from active list
```

---

4. **Strategy Decision Loop**

- React to the current top-of-book prices.
- Example condition: "If best bid > 100.00, sell 10 shares"
- Logic must:
  - Check conditions
  - Place order via `OrderManager`
  - Respond to execution feedback

---

5. **No Memory Leaks**

- All dynamic memory must be owned and cleaned up appropriately.
- Use tools such as:

  - AddressSanitizer (`-fsanitize=address`)
  - Valgrind (if running on Linux)
- If running the program does not return cleanly, memory issues must be addressed.

🔐 Memory Safety Expectations

Memory management in this phase must be **explicit, responsible, and modern**:

You must:

- Use **RAII (Resource Acquisition Is Initialization)** to manage resource lifetimes
- Store heap-allocated objects in `std::unique_ptr` or `std::shared_ptr` **only if shared ownership is necessary**
- Avoid any manual `new` or `delete` calls
- Handle all object lifetimes through scope-bound containers (e.g., STL containers or smart pointers)
- Remove elements from containers when they are no longer active (e.g., filled orders, empty price levels)

You must not:

- Leak memory by leaving unused objects in memory
- Access invalidated memory (use-after-free)
- Double-delete anything
- Use raw pointers to manage ownership unless you are wrapping them in smart pointers immediately

---

📦 Deliverables

Your submission should include the following files and directories:

📁 `market_snapshot.h / .cpp`

- A class that maintains bid/ask levels
- Provides `get_best_bid()` and `get_best_ask()`

- Uses `std::unique_ptr` to manage each `PriceLevel`
- Properly removes levels when quantity reaches 0

📁 `order_manager.h / .cpp`

- A class that tracks your placed orders
- Stores each `MyOrder` in a `std::unique_ptr`
- Supports: place, cancel, handle fill, print status

📁 `main.cpp`

- Contains your strategy logic
- Loops over a simulated feed
- Updates the market snapshot and handles execution events
- Places orders in reaction to market data

📁 `sample_feed.txt`

- Simulated market feed to test your system
- Each line is either a BID/ASK update or an execution fill

📁 `output.log` *(optional but encouraged)*

- Log of your program's actions: price updates, order placements, fills

📁 `README.md`

Include the following:

- Overview of your system architecture
- Explanation of how memory is managed safely
- Instructions on how to compile and run the code
- How to verify correctness and memory safety

📁 Example Input File: `sample_feed.txt`

```
BID 100.10 300
ASK 100.20 250
```

```
BID 100.10 0
ASK 100.25 200
EXECUTION 1 10
EXECUTION 1 40
```

Explanation:

- Starts with an active bid/ask pair
- Bid at 100.10 disappears
- Execution fills 10, then 40 more from order ID 1

🖨️ Sample Output File: `output.log`

```
[Market] Best Bid: 100.10 x 300
[Market] Best Ask: 100.20 x 250
[Market] Best Bid: 100.10 removed
[Market] New Ask: 100.25 x 200

[Strategy] Placing SELL order at 100.20 x 50 (ID = 1)
[Execution] Order 1 filled: 10
[Order] Order 1 partially filled: 10 / 50
[Execution] Order 1 filled: 40
[Order] Order 1 completed (50 / 50) and removed
```

✅ Evaluation Checklist

Requirement	Criteria	Score
Market snapshot updates	Correctly tracks best bid/ask	✅
Memory-safe order book	Uses `unique_ptr`, no leaks	✅
Smart pointer usage	No raw `new` or `delete`	✅
Order tracking system	Correctly tracks, fills, and cancels	✅
Strategy loop	Places orders based on logic	✅
Clean output logs	Human-readable trade info	✅
README.md	Architecture, how to run, how it works	✅

📁 `sample_feed.txt`

```
# Initial market setup
BID 100.10 300
ASK 100.20 250

# New bid comes in at a better price
BID 100.15 200
```

```
# Ask gets updated to a better price
ASK 100.18 100

# Partial quantity removed from top ask
ASK 100.18 60

# Bid at 100.10 disappears (canceled or filled)
BID 100.10 0

# Ask quantity reduced again
ASK 100.18 30

# Ask completely gone
ASK 100.18 0

# New ask comes in higher
ASK 100.25 200

# Strategy places an order (we assume ID = 1)
# (This part would be handled by your program, not the feed)

# Simulated execution events from the "exchange"
EXECUTION 1 10
EXECUTION 1 20

# New bid comes in
BID 100.17 100

# Another strategy order placed (assume ID = 2)

# Fill for that new order
EXECUTION 2 50

# Bid at 100.15 reduced in quantity
BID 100.15 100

# Ask comes in at a new level
ASK 100.22 150

# Execution that completes an order fully
EXECUTION 2 50

# New top bid
BID 100.20 300

# Final snapshot change
ASK 100.23 100
```

Here's a **simple and extensible C++ parser** for `sample_feed.txt` that reads bid/ask updates and execution events into a structured format. This parser is designed to integrate cleanly with your `MarketSnapshot` and `OrderManager`.

feed_parser.h

```
#pragma once

#include <string>
#include <vector>
#include <fstream>
#include <sstream>
#include <iostream>

enum class FeedType {
    BID,
    ASK,
    EXECUTION,
    UNKNOWN
};

struct FeedEvent {
    FeedType type = FeedType::UNKNOWN;
    double price = 0.0;
    int quantity = 0;
    int order_id = -1; // used for EXECUTION only

    // Debug print
    void print() const {
        switch (type) {
            case FeedType::BID:
                std::cout << "[BID] " << price << " x " << quantity << "\n";
                break;
            case FeedType::ASK:
                std::cout << "[ASK] " << price << " x " << quantity << "\n";
                break;
            case FeedType::EXECUTION:
                std::cout << "[EXECUTION] Order " << order_id << " filled: " << quantity << "\n";
                break;
            default:
                std::cout << "[UNKNOWN]\n";
        }
    }
};

std::vector<FeedEvent> load_feed(const std::string& filename) {
    std::ifstream file(filename);
    std::vector<FeedEvent> events;

    if (!file.is_open()) {
        std::cerr << "Error: could not open file " << filename << "\n";
        return events;
    }

    std::string line;
    while (std::getline(file, line)) {
        // Ignore comments and empty lines
        if (line.empty() || line[0] == '#') continue;

        std::istringstream iss(line);
        std::string type;
        iss >> type;

        if (type == "BID") {
            double price;
            int qty;
            if (iss >> price >> qty) {
                events.push_back({FeedType::BID, price, qty});
            }
        } else if (type == "ASK") {
```

```
            double price;
            int qty;
            if (iss >> price >> qty) {
                events.push_back({FeedType::ASK, price, qty});
            }
        } else if (type == "EXECUTION") {
            int order_id;
            int filled;
            if (iss >> order_id >> filled) {
                events.push_back({FeedType::EXECUTION, 0.0, filled, order_id});
            }
        } else {
            std::cerr << "Unknown event type: " << line << "\n";
        }
    }

    return events;
}
```

✅ How to Use It

📁 In `main.cpp`

```
#include "feed_parser.h"

int main() {
    auto feed = load_feed("sample_feed.txt");

    for (const auto& event : feed) {
        event.print();

        // Integrate with your components:
        if (event.type == FeedType::BID) {
            snapshot.update_bid(event.price, event.quantity);
        } else if (event.type == FeedType::ASK) {
            snapshot.update_ask(event.price, event.quantity);
        } else if (event.type == FeedType::EXECUTION) {
            order_manager.handle_fill(event.order_id, event.quantity);
        }
    }

    return 0;
}
```

Keep in mind, this submission will count for everyone in your Q22026 Group group.

Choose a submission type

Text | Upload | More

Submit Assignment